Universidad Nacional de Colombia
Facultad de Ingeniería

Ejercicios

Presentado por:
Leal Diaz Layonel Camilo
Jaekle Till

Profesor:
José J. Martínez P.

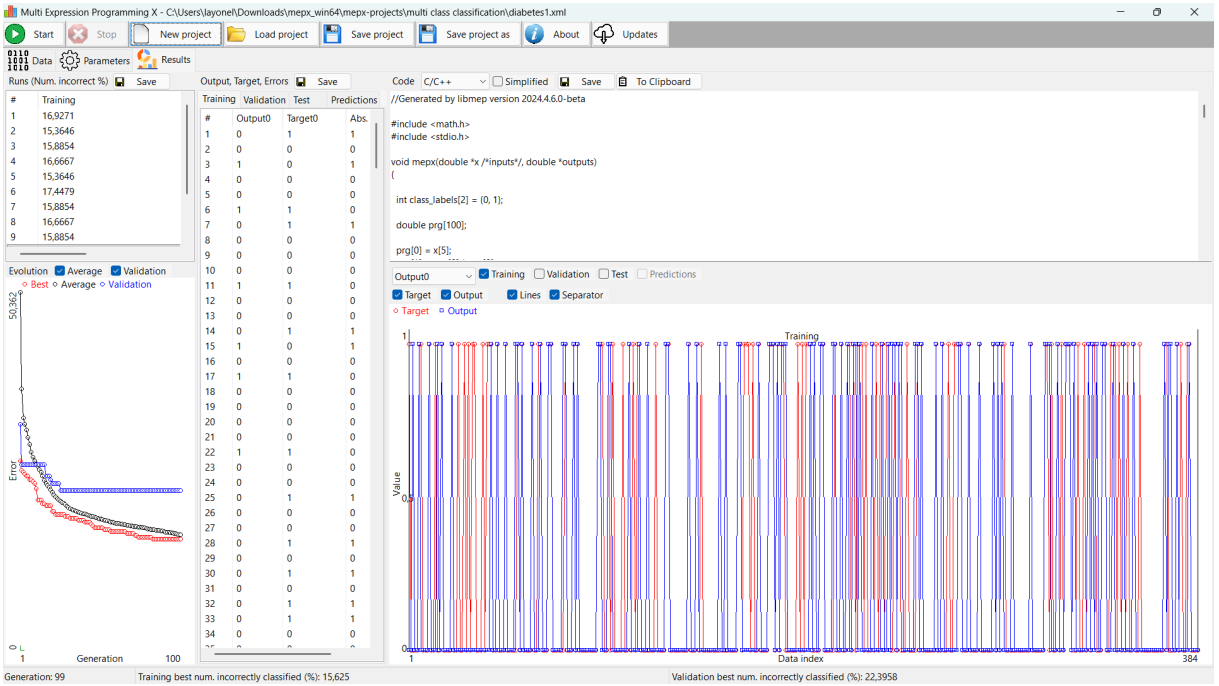
Asignatura:
Inteligencia Artificial y mini-robots

Bogotá, D.C

5 de marzo de 2026

Ejercicio 1.

Descargue MEPX, estúdielo y corra uno de los ejemplos que trae.



Se revisó el ejemplo de clasificación multiclase de diabetes1

Ejercicio 2. Codificador7Segmentos.ipynb

Resumen del Proceso Algorítmico

1. **Modelado Estructural**: Se implementó un sistema de **Programación Genética (PG)** donde, a diferencia de los algoritmos tradicionales, los cromosomas no son listas fijas sino **árboles ejecutables**. El conjunto de funciones se limitó a operadores lógicos puros (AND, OR, NOT) y los terminales a las entradas binarias (A, B, C, D).
2. **Evolución Estocástica**: El motor evolutivo utilizó el método de *Half-and-Half* para generar diversidad inicial en la arquitectura de los circuitos. A través de **cruce de un punto en ramas** y mutación uniforme, el algoritmo exploró diferentes topologías de compuertas hasta encontrar la combinación correcta.
3. **Evaluación de Aptitud (Fitness)**: La aptitud se definió de forma discreta basándose en el éxito sobre la tabla de verdad del codificador. Cada circuito fue compilado y ejecutado como un programa de Python para validar su salida frente a las 10 combinaciones posibles (dígitos 0-9).

Ejercicio 3.

Resumen de Mejoras y Cambios Técnicos

Corrección de Estabilidad (Bug Fix):

Se solucionó un error crítico de tipo AttributeError en la función de Cruce (Crossover). El problema radicaba en una inconsistencia de nomenclatura donde el algoritmo intentaba acceder a propiedades inexistentes durante el intercambio de ramas. Se estandarizó la estructura de la clase Nodo (usando .valor y .hijos) para garantizar que la recombinación genética sea fluida y sin interrupciones.

Optimización del Motor de Ejecución:

Se refactorizó el motor de simulación para que la evaluación de los 300 individuos por generación fuera más ligera. Se simplificó la lógica de movimiento y entrega, permitiendo que el algoritmo explore miles de combinaciones de "ADN" en pocos segundos, lo que facilita alcanzar puntuaciones altas (superiores a 800 puntos) en menos de 20 generaciones.

Lógica Sensorial Mejorada:

Se integró el terminal inteligente Mirar_Hacia_Ingeniero. Esta función actúa como un "radar" que permite al robot orientarse automáticamente hacia el objetivo más cercano que aún no ha recibido su galleta. Esta mejora redujo drásticamente el tiempo de convergencia, ya que el robot no depende únicamente del azar para encontrar a los ingenieros en la sala.

Visualización Robusta y Universal:

Se migró el sistema visual a formas geométricas estándar de Matplotlib para generar el video del resultado final. El robot ahora es un Triángulo azul que rota según su dirección (^, >, v, <). Los ingenieros se representan con Cuadrados rojos (esperando) y Círculos verdes (alimentados).

Interactividad Dinámica:

Se implementó un reproductor de video basado en FuncAnimation y HTML. A diferencia de los gráficos estáticos previos, esta mejora permite observar la ejecución del mejor programa en tiempo real, con controles de reproducción (play, pausa y velocidad) para analizar el comportamiento lógico del robot paso a paso.

Conclusión del Algoritmo

Se logró un sistema de Programación Genética capaz de sintetizar comportamientos complejos de navegación a partir de instrucciones simples. Aunque la lógica generada por el algoritmo puede parecer caótica al leerse (árboles profundos), el resultado es altamente efectivo, logrando una cobertura casi total de los ingenieros en la sala cuadrada. La transición a formas geométricas aseguró que la herramienta sea lúdica y funcional en cualquier entorno de simulación.